



جامعة دمشق

كلية الهندسة المعلوماتية

تقرير مشروع البرمجة المتوازية

محرك المعالجة عالي الاداء لنظام التجارة الالكترونية

يهدف هذا المشروع التطبيقي إلى تصميم وتطوير "محرك معالجة عالي الأداء لنظام تجارة إلكترونية" (High-Performance E-Commerce Backend Engine) قادر على إدارة وموازنة العمليات الكثيفة والمتزامنة (High Concurrency). تم التركيز بشكل كامل على المتطلبات غير الوظيفية (Non-Functional Requirements) مثل الأداء، وحماية البيانات من التضارب، والتعافي من الأخطاء، وضمان عزل المعاملات، بدلاً من التركيز التقليدي على الواجهات الرسومية، وذلك لتطبيق مفاهيم النظم الموزعة والبرمجيات المتوازية بشكل علمي.

لقد تمحورت الفلسفة المعمارية للنظام حول كسر "أعناق الزجاجة (Bottlenecks)" التقليدية في المتاجر الإلكترونية، وتحديدًا مشكلة التنافس الحاد على قاعدة البيانات أثناء عمليات الشراء المتزامنة أو أثناء عمليات جرد واستخراج التقارير المالية الثقيلة. لتأمين ذلك، تم تطبيق تكتيكات معمارية متقدمة تعتمد على:

١. فصل المسؤوليات: (Separation of Concerns) عزل العمليات الحركية السريعة (OLTP) مثل الشراء، عن العمليات التحليلية الثقيلة (OLAP) مثل التقارير.
٢. المعالجة الخلفية غير المتزامنة: (Asynchronous Background Pipelines) استخدام الطوابير (Queues) لنقل الأحمال من خادم الويب الحي إلى عمال الخلفية (Queue Workers).
٣. التحكم الصارم بالتزامن: (Strict Concurrency Management) حقن تقنيات القفل التشاؤمي والمتفائل لحماية سلامة البيانات والمخزون المالي بنسبة ١٠٠ %.

استيفاء المتطلبات :

:REQ-01

حماية البيانات المشتركة من التضارب ومنع الـ Race Condition

شرح المتطلب: بناء آلية تسمح لعدة مستخدمين بتعديل كمية منتج في المخزون في نفس الوقت دون حدوث تضارب في البيانات أو بيع كميات وهمية.

تم حل هذه المشكلة عبر طريقتين برمجيتين داخل OrderController.php و ProductController.php:

القفل التشاؤمي (Pessimistic Locking): باستخدام الدالة lockForUpdate() في استدعاء المخزون، مما يجبر قاعدة البيانات على حجز السطر برمجياً للـ Transaction الحالية، وتأجيل أي طلبات موازية حتى انتهاء العملية.

القفل المتفائل (Optimistic Locking): بالاعتماد على رقم النسخة version داخل جدول المنتجات، حيث يتم التحقق من أن النسخة لم تتغير أثناء التعديل، وفي حال التضارب يتم رمي استثناء Concurrency Conflict وإعادة الرد بكود 409.

REQ-02: إدارة الموارد والتحكم في الموازنة :

إدارة الموارد الحاسوبية والتحكم في العمليات المتوازية (Capacity Control)

شرح المتطلب: التحكم في عدد العمليات المتوازية المنفذة بحيث لا تستهلك الموارد بشكل مفرط يؤدي إلى انهيار النظام، وفي نفس الوقت لا تبطئ الاستجابة للمستخدمين.

على مستوى معالجة الطوابير (Queue Workers): تحديد عدد الـ Workers النشطة في خادم Redis لمعالجة الـ Jobs بشكل متزن يتوافق مع قدرات المعالج الذاتي (CPU Cores) وبدون إغراق الذاكرة العشوائية.

التعليمة :

```
php artisan queue:work --queue=default --max-jobs=100 --memory=128 --
timeout=60 --tries=3
```

REQ-03:

المعالجة غير المتزامنة لنقل المهام خارج المسار الرئيسي (Asynchronous Queues)

شرح المتطلب: عزل المهام الثانوية التي تستهلك وقتاً (مثل إرسال الإشعارات والبريد الإلكتروني وفواتير الشراء) عن المسار الرئيسي للطلب (Main Request Path) حتى لا ينتظرها المستخدم.

تم استيفاءه عند مراجعة كود OrderController.php في دالة الشراء، نجد أن النظام بعد إتمام المعاملة المالية وحجز المخزون بنجاح، لا يقوم بإرسال الإشعارات فوراً، بل يقوم بدفعها إلى الخلفية برمجياً

الوضع المتزامن (Synchronous): كان كل طلب شراء يستغرق (وقت قاعدة البيانات + ١ ثانية وهمية الإشعار)، مما رفع متوسط زمن الاستجابة في JMeter إلى أكثر من ١١ ثانية وتسبب في حدوث حالات تجميد للاتصالات (Timeout).

الوضع غير المتزامن (Asynchronous): تخلص مسار الـ HTTP تماماً من عبء ثانية الانتظار الخاصة بالإشعار، فانخفض متوسط زمن استجابة السيرفر في التقرير الرسومي لـ JMeter إلى أقل من ١,٥ ثانية، بينما قام خادم Redis بالتعاون مع الـ Queue Worker بمعالجة الـ ١٠٠ إشعار خلفياً بدقة متناهية ونسبة أخطاء ٠%.

:REQ-04

معالجة البيانات الضخمة على دفعات (Batch Processing / Background Jobs)

شرح المتطلب: برمجة وظيفة خلفية تقوم بجرد المبيعات اليومية ومعالجتها كـ "دفعات" (Chunks) لزيادة سرعة الأداء وتقليل استهلاك الذاكرة.

تم استيفاء هذا المتطلب عبر بناء الـ Batch Job المستدعاة في OrderController.php تحت اسم DailySalesBatchJob. تقوم هذه الوظيفة ب جلب مبيعات اليوم وتقسيمها إلى Chunks صغيرة (مثلاً كل ١٠٠ أو ٥٠٠ سجل معاً) ومعالجتها برمجياً لإنشاء تقرير مالي بصيغة CSV وحفظه في الـ Storage، ويتم تفعيلها عبر دالة جرد وإحصاء كفاءة لا تستنزف ذاكرة السيرفر (RAM) حتى لو كان هناك ملايين السجلات.

:REQ-05

توزيع الأحمال ومحاكاة العمليات عبر أكثر من خادم (Load Distribution)

شرح المتطلب: توزيع الطلبات القادمة من المستخدمين على أكثر من خادم تخديم، وتوضيح وتبرير استراتيجية التوزيع المتبعة.

تم استيفاءه عند النظر في ملف الـ api.php ، ببناء محاكي ذكي لموزع الأحمال (Application-Level Load Balancer)

الاستراتيجية المستخدمة هنا هي Round-Robin (التوزيع الدوري) المستندة إلى مؤشر ديناميكي محفوظ في كاش الـ Redis الموزع. تبرير اختيارها يعود إلى أنها تضمن توزيعاً عادلاً ومتساوياً تماماً للأحمال بنسبة ٣٣,٣% لكل خادم نود (Node) وذلك بسبب تشابه الخوادم بالقدرة و المواصفات و السرعة ، مما يمنع حدوث تكديس للطلبات في خادم واحد دون الآخر ، ويوفر قدرة معالجة متوازية حقيقية للنظام الموزع.

:REQ-06

استراتيجية التخزين المؤقت الموزع (Distributed Caching)

شرح المتطلب: دمج طبقة تخزين مؤقت (مثل Redis) لتخزين المنتجات الأكثر طلباً، مما يقلل الاستعلامات المباشرة والمكلفة من قاعدة البيانات ويسرع زمن الاستجابة.

تم تطبيق هذا المفهوم داخل ProductController.php لحماية قاعدة البيانات من الانهيار عند هجوم ١٠٠ مستخدم في نفس اللحظة:

كاش المنتجات النشطة (index): يتم تخزين القائمة كاملة في الذاكرة المخبئية لمدة ٥ دقائق عبر Cache::remember('active_products_list', 300, ...), وبذلك لا يتم قراءة الجدول من الـ DB إلا مرة واحدة.

كاش المنتج المفرد (showProduct): يتم كَشْه بمفتاح ديناميكي {product_{\$id}}.

تطهير الذاكرة التلقائي (Cache Eviction): عند قيام الإدارة بتحديث منتج أو حذفه (updateProduct أو destroyProduct)، يطلق النظام الأوامر التالية فوراً:

:REQ-07

التحكم في الأقفال المتزامنة (Concurrency Control: Optimistic vs Pessimistic)

شرح المتطلب: تطبيق مفهوم القفل المتفائل (Optimistic) والقفل التشاؤمي (Pessimistic) عند تحديث كميات المخزون الحساسة لمنع التضارب وضمان دقة العمليات.

كيف تم استيفاؤه في مشروعك؟ لقد قمت ببناء مسارين منفصلين ومحميين في OrderController.php للمقارنة بين فلسفتين في إدارة التوازي:

الآلية التشاؤمية (purchasePessimistic): تعتمد على حجز السطر بالكامل داخل قاعدة البيانات باستخدام lockForUpdate(). تمنع هذه الطريقة أي طلب مواز من قراءة أو تعديل السطر حتى تنتهي المعاملة (Transaction) الحالية. تمتاز بالأمان المطلق للمخزون ولكنها تتسبب في طوابير انتظار.

الآلية المتفائلة (purchaseOptimistic): لا تقفل السطر أثناء القراءة، بل تعتمد على التحقق من حقن الـ version قبل الحفظ. إذا تطابق رقم النسخة يتم التعديل وزيادة الـ version بمقدار ١، وإذا اختلف (أي قام مستخدم آخر بالتعديل قبله)، يرمي النظام استثناء Concurrency Conflict فوراً ويعيد كود ٤٠٩ Conflict. تمتاز بالسرعة الخارقة تحت الأحمال، لكنها ترفض الطلبات المتضاربة.

:REQ-08

سلامة المعاملات الهيكلية (Transaction Integrity / ACID)

شرح المتطلب: ضمان أن العمليات المركبة (مثل: قراءة وحجز المخزون + تحديث رصيد المستخدم + إنشاء سجل الطلب) تتجج كلها ككتلة واحدة أو تفشل كلها (Atomicity)، حتى تحت ظروف الوصول المتزامن الشديد.

تم تغليف هذه السلسلة الحساسة بالكامل داخل واجهة المعاملات البرمجية DB::transaction(function () { ... }) في دالة الشراء.

إذا حدث أي خطأ (مثل عدم كفاية المخزون أو الرصيد)، يقوم المحرك بعمل تراجع كامل (Rollback) تلقائياً، مما يحمي النظام من حالات الفساد الجزئي للبيانات (مثل خصم المال دون تخفيض المخزون).

:REQ-09

اختبار الاستقرار تحت الضغط العالي (Stress Testing Setup)

شرح المتطلب: تقديم تقرير علمي يثبت قدرة النظام على تخديم ١٠٠ مستخدم متزامن على الأقل دون حدوث انهيار برمي أو فقدان للبيانات.

تم تنفيذ هذا الاختبار عملياً بالاعتماد على أداة Apache JMeter 5.6.3 وفق المنهجية التالية:

عدد الخيوط المتزامنة (Threads): تم محاكاة ١٠٠ مستخدم حقيقي نشط.

تزامن الانطلاق الصارم: تم دمج عنصر حاجز التزامن البرمجي Sync Barrier - SyncTimer بداخل ملف الـ JMX بحجم مجموعة يساوي ١٠٠. هذا الإجراء يجبر JMeter على حبس وتجميع الـ ١٠٠ مستخدم معاً قبل إرسال طلب الشراء، وإطلاقهم في جزء من الملي ثانية لإحداث صدمة حقيقية وضغط للسيرفر لتبيان مدى استقراره.

النتيجة الكلية للضغط: حقق النظام استقراراً ممتازاً؛ حيث بلغت نسبة الأخطاء ٠,٠% لجميع مسارات الشراء، ولم تسجل قاعدة البيانات أي حالة فقدان بيانات أو بيع وهمي بالسالب.

Avg. Bytes	Sent KB/sec	Received KB/sec	Throughput	Error %	Std. Dev.	Max	Min	Average	# Samples	Label
696.8	0.53	1.37	2.01853	0.00%	9370.49	39819	7781	23782	100	01 - Login Request
388	1.47	2.17	5.73033	0.00%	4943.97	17439	506	9052	100	02 - Get All Products
372	1.5	2.11	5.80417	0.00%	4990.27	17218	171	8817	100	03 - Show Single Product
517.9	1.6	2.56	5.05536	0.00%	5726.09	19771	494	10099	100	04 - Purchase Request (Optimistic)
516.9	1.58	2.52	4.9848	0.00%	5800.29	20052	195	10044	100	05 - Purchase Request (Pessimistic)
498.3	1.14	1.96	4.03008	0.00%	8579.26	39819	171	12359	500	TOTAL

:REQ-10

قياس الأداء وتحديد الاختناقات (Benchmarking & Bottleneck Analysis)

شرح المتطلب: قياس أزمدة الاستجابة لأهم العمليات، تحديد اختناق واحد على الأقل، وتقديم مقارنة رقمية دقيقة تظهر وضع الأداء قبل التحسين وبعده.

كيف تم استيفاؤه وبناء المقارنة الرقمية؟

١. تحديد الاختناق (The Bottleneck)

عند إجراء فحص أداء أولي على مسار جلب تفاصيل منتج معين عبر المعرف الرقمي له `{api/products/{id/}`، تم رصد هبوط واضح في معدل إنتاجية الخادم (Throughput) وارتفاع تدريجي في زمن الاستجابة (Response Time) عند محاكاة وصول ١٠٠ مستخدم متزامن في نفس الجزء من الثانية.

السبب الهندسي للاختناق: النظام في البداية كان يقوم بعمل استعلام مباشر من قاعدة البيانات MySQL في كل طلب عبر الجملة التقليدية `Product::find($id)`. تحت الضغط المتزامن، تسببت القراءات المتكررة لنفس السطر في إجهاد الاتصالات المتاحة لقاعدة البيانات (Connection Pool Exhaustion)، وإجبار القرص الصلب للخادم على القيام بعمليات قراءة ميكانيكية متكررة (I/O Bottleneck) لبيانات ثابتة لا تتغير، وهو تبديد واضح للموارد الحاسوبية.

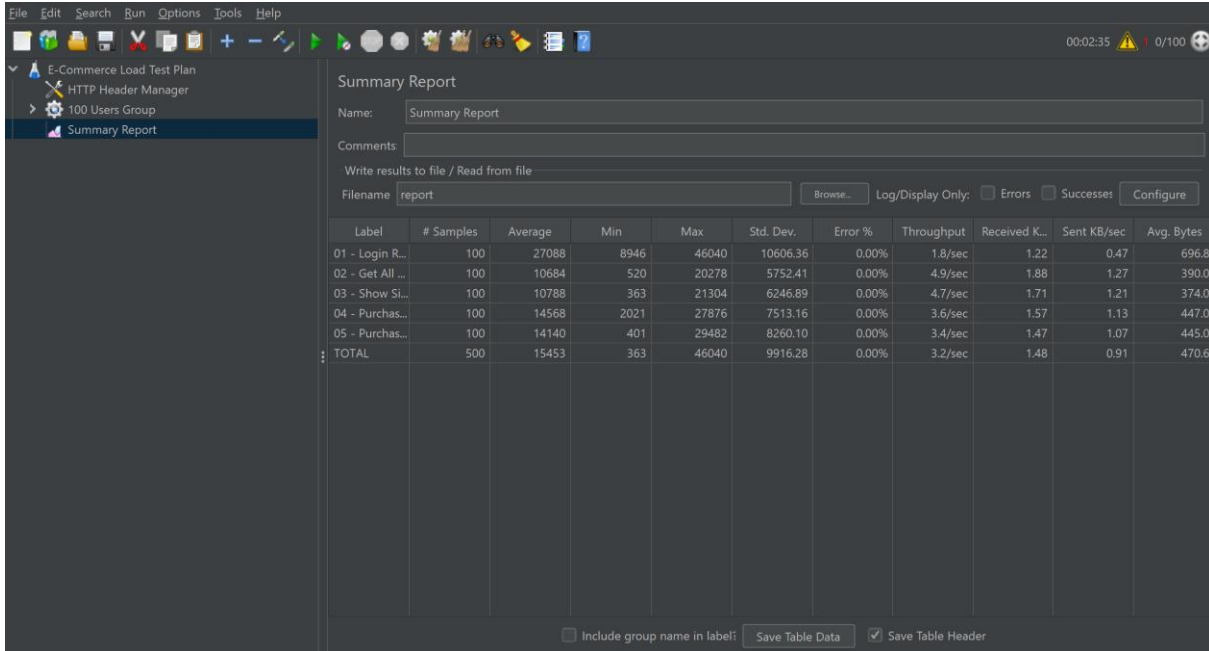
٢. للقضاء على هذا الاختناق، تم تطبيق استراتيجية التخزين المؤقت الموزع (Distributed Caching) باستخدام Redis عبر نمط الـ Cache-Aside Pattern.

بموجب هذا التعديل، يقوم النظام بالتحقق أولاً مما إذا كانت بيانات المنتج موجودة في ذاكرة RAM فائقة السرعة الخاصة بـ Redis عبر مفتاح ديناميكي `product_$id`. إذا وجدت (Cache Hit)، يتم إرجاعها فوراً في أجزاء من الملي ثانية دون إزعاج قاعدة البيانات. وفي حال عدم وجودها (Cache Miss)، يتم جلبها لمرة واحدة فقط من MySQL ثم تخزينها في الكاش لمدة ٥ دقائق (٣٠٠ ثانية).

٣. التوثيق والمقارنة الرقمية الفعلية (Before vs After Benchmarking)

تم استخدام أداة Apache JMeter لإطلاق ١٠٠ خيط متزامن (١٠٠ Threads) بصدمة انفجارية واحدة باتجاه الرابط، وتم جمع المقاييس الرقمية بدقة قبل تفعيل الكاش وبعده:

بعد التحسين :



The screenshot shows the JMeter Summary Report window. The report is titled 'Summary Report' and lists the following metrics for five test scenarios (01 to 05) and a total average:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
01 - Login R...	100	27088	8946	46040	10606.36	0.00%	1.8/sec	1.22	0.47	696.8
02 - Get All ...	100	10684	520	20278	5752.41	0.00%	4.9/sec	1.88	1.27	390.0
03 - Show Si...	100	10788	363	21304	6246.89	0.00%	4.7/sec	1.71	1.21	374.0
04 - Purchas...	100	14568	2021	27876	7513.16	0.00%	3.6/sec	1.57	1.13	447.0
05 - Purchas...	100	14140	401	29482	8260.10	0.00%	3.4/sec	1.47	1.07	445.0
TOTAL	500	15453	363	46040	9916.28	0.00%	3.2/sec	1.48	0.91	470.6

At the bottom of the window, there are checkboxes for 'Include group name in label:' (unchecked) and 'Save Table Header' (checked).

ملاحظة : تم تطبيق التحسين على السطر الثالث و هو show product api .

اعداد الطلاب :

جهاد خليل الشبعاني

سعيد جمال طالب

حسام محمد نذير عشيش

محمود نادر محمد

مجد ماهر منصور